

Test 2 Practice — Answer Key

Multiple Choice Answers

1. D — R7. JSR stores PC+1 (the return address) in R7.
 2. B — Decrement R6, then store. Stack grows downward: decrement the pointer first to claim the slot, then store.
 3. C — The address of LABEL. LEA loads the effective address itself, not the value stored there.
 4. B — The address of a null-terminated string. PUTS reads characters starting at that address until it finds x0000.
 5. B — Not zero (either negative or positive). BRnp branches when the N or P condition code is set (i.e., the result was anything other than zero).
 6. B — Save R1 and R2 on the stack and restore them before RTI. The hardware does not save general-purpose registers on TRAP entry.
 7. C — Loads the value stored at address x4000. LDI is double-indirect: it reads the address from PTR (x4000), then reads the value at x4000.
 8. C — x0200 or higher in OS/supervisor space (e.g., x0300). Handler code must be in supervisor space, not in the trap vector table or user program area.
 9. C — `JMP R7`. RET is an assembler pseudo-op that expands to `JMP R7`.
 10. C — TRAP x20 (GETC). GETC reads one character, does not echo, and returns it in R0. TRAP x23 (IN) also reads but echoes and shows a prompt.
-

Assembly Language Answers

Problem 1 — Print "LC3" using OUT and a loop

LC-3 Solution:

```

; Print each char of string MSG
; Using OUT
; Loop until NUL terminator is found
; DO NOT use PUTS

        .orig    x3000
        BR start
MSG      .stringz "LC3"

start
        LEA R1, MSG
loop     LDR R0, R1, #0
        BRz exit
        OUT
        ADD R1, R1, #1
        BR loop
exit     HALT
        .end

```

Problem 2 — Count non-zero elements

LC-3 Solution:

```

; Count the number of non-zero elements
; in in 6 element array VALUES
; Store count in NONZERO_COUNT

        .orig    x3000
BR start
VALUES  .fill #-3
        .fill #7
        .fill #0
        .fill #5
        .fill #-1
        .fill #4
count   .fill #6
NONZERO .fill #0

start
        LEA R1, VALUES
        LEA R2, NONZERO
        AND R3, R3, #0
        LD R4, count

loop    LDR R0, R1, #0
        BRz next
        ADD R3, R3, #1
next    ADD R1, R1, #1
        ADD R4, R4, #-1
        BRp loop
        STR R3, R2, #0
exit    HALT
        .end

```

Expected result: `NONZERO_COUNT = 5` (-3, 7, 5, -1, and 4 are non-zero; 0 is skipped).

Problem 3 — Subroutine ABS_VAL

LC-3 Solution:

```

; Write subr called ABS_VAL, test value in R0
; returns absolute value in R0
; Save and restore any reg other than R0
; init stack
; load val #-5
; use JSR
; store result in RESULT

        .orig    x3000
        BR start
RESULT  .fill 0
STACK   .fill x4000
VALUE   .fill #-5

start
        LD R6, STACK
        LD R0, VALUE
        JSR ABS_VAL
        ST R0, RESULT
        HALT

ABS_VAL
        ADD R0, R0, #0
        BRzp pos
        NOT R0, R0
        ADD R0, R0, #1

pos
        RET

exit    HALT
        .end

```

Expected result: RESULT = 5

Problem 4 — Trap ASCII at vector x0027

LC-3 Solution:

```

; Write a trap
; converts value in R0 to ASCII
; Add x30 to R0 to convert
; trap x27
; handler at x300
; load 5 into R0, trap the store in ASCII

        .orig    x27
        .fill    TRAP_ASCII
        .end

        .orig    x300
TRAP_ASCII
        ADD R6, R6, #-1
        STR R1, R6, #0
        LD R1, convert
        ADD R0, R0, R1
        LDR R1, R6, #0
        ADD R6, R6, #1
        RTI
convert .fill x30
        .end

        .orig    x3000
        BR start
ASCII .fill 0
VALUE .fill #9
STACK .fill x4000

start
        LD R6, STACK
        LD R0, VALUE
        TRAP x27
        ST R0, ASCII

        HALT

        .end

```

Expected result: ANSWER = 39

Debugging Answers

D1 — Access Violation on stack push

Bug: R6 (the stack pointer) is never initialized. It starts at 0. When `DOUBLE` executes `ADD R6, R6, #-1`, R6 wraps around to `xFFFF`. `STR R1, R6, #0` then attempts to write to address `xFFFF`, which is inside the device register region (`xFE00-xFFFF`). Device registers are supervisor-only — user-mode code cannot write to them → Access Violation.

Fix: Initialize R6 to a safe stack area before calling `DOUBLE`:

```
LD  R6, STK_INIT
...
STK_INIT  .FILL  x4000
```

D2 — Wrong POP order (registers swapped)

Bug: The stack is LIFO — last in, first out. R0 was pushed first, then R1. R1 is therefore on top (at the address R6 points to after both pushes). The code pops into R0 first (getting R1's value) and then pops into R1 (getting R0's value), swapping them.

Fix: Pop in reverse order of push — pop R1 first, then R0:

```
LDR  R1, R6, #0
ADD  R6, R6, #1      ; pop R1 first (it was pushed last)
LDR  R0, R6, #0
ADD  R6, R6, #1      ; pop R0 second
```

D3 — Nested JSR overwrites R7

Bug: When `JSR INNER` executes inside `OUTER`, R7 is overwritten with the address of `OUTER`'s `RET` instruction. The original caller's return address (placed in R7 by `JSR OUTER`) is lost. When `INNER`'s `RET` fires, it returns to `OUTER`'s `RET`. `OUTER`'s `RET` then jumps to its own address — an infinite loop. `OUTER` never returns to its original caller.

Fix: Save R7 before calling `JSR INNER`, then restore it afterward:

```
OUTER      ADD  R6, R6, #-1
           STR  R7, R6, #0      ; save caller's return address
           ST   R2, SAVE_R2
           ADD  R2, R0, R0
           JSR  INNER
           LD   R2, SAVE_R2
           LDR  R7, R6, #0
           ADD  R6, R6, #1      ; restore return address
           RET
```

D4 — BRzp loops one extra iteration

Bug: `BRzp` branches when the count (R1) is zero **or** positive. After 5 iterations, `R1 = 0`, `CC = Z`. `BRzp` is taken and the loop reads a 6th element past the end of the array. The loop does not stop until R1 goes negative (after a 6th iteration), producing an incorrect sum.

Fix: Change `BRzp LOOP` to `BRp LOOP`. `BRp` branches only when `R1 > 0`, so the loop stops as soon as the count reaches zero.